SC1001: Introduction to Computational Thinking

Tutorial 1: Flowcharts, Pseudocode, Data Types, and Variables

Jordan Boyd-Graber

Nanyang Technological University

Me

- Professor in CCDS
- Formerly at the University of Maryland and the University of Colorado
- Research: natural language processing, question answering, topic models, and human–AI interaction
- Born in Colorado; grew up in Iowa
- High school in Arkansas
- Undergraduate study in California
- Graduate school in New Jersey
- Worked on an electronic dictionary in Berlin and Google Books in New York
- You can call me Jordan, JBG, or Boyd-Graber

What We Are Practising Today

- Trace and repair an algorithm
- If statements
- FizzBuzz
- Distinguish Python expressions, assignments, and errors
- Design an algorithm before writing Python
- Use a simple heuristic when an exact solution is unnecessary

Discussion 1: What Is Wrong?

The algorithm should read 10 scores and compute their average.

```
Initialize student_counter to zero
While student_counter is less than or equal to ten
    Input the next score
    Add the score into the total
EndWhile
Set class_average to total divided by ten
```

Find every variable or loop-control error before proposing a repair.

Discussion 1: What Is Wrong?

The algorithm should read 10 scores and compute their average.

```
Initialize student_counter to zero
While student_counter is less than or equal to ten
    Input the next score
    Add the score into the total
EndWhile
Set class_average to total divided by ten
```

What was this originally?

Discussion 1: What Is Wrong?

The algorithm should read 10 scores and compute their average.

```
Initialize student_counter to zero
While student_counter is less than or equal to ten
    Input the next score
    Add the score into the total
EndWhile
Set class_average to total divided by ten
```

Will this loop end?

Discussion 1: A Corrected Algorithm

```
Set total to 0
Set student_counter to 1
While student_counter <= 10
    Input score
    Set total to total + score
    Set student_counter to student_counter + 1
EndWhile
Set class_average to total / 10
```

Three issues

Initialise before use

Discussion 1: A Corrected Algorithm

```
Set total to 0
Set student_counter to 1
While student_counter <= 10
    Input score
    Set total to total + score
    Set student_counter to student_counter + 1
EndWhile
Set class_average to total / 10
```

Three issues

Process exactly ten scores

Discussion 1: A Corrected Algorithm

```
Set total to 0
Set student_counter to 1
While student_counter <= 10
    Input score
    Set total to total + score
    Set student_counter to student_counter + 1
EndWhile
Set class_average to total / 10
```

Three issues

Update the loop-control variable

Discussion 1: An Alternate Version

```
Set total to 0
Set student_counter to 0
While student_counter < 10
    Input score
    Set total to total + score
    Set student_counter to student_counter + 1
EndWhile
Set class_average to total / student_counter
```

Discussion 1: An Alternate Version

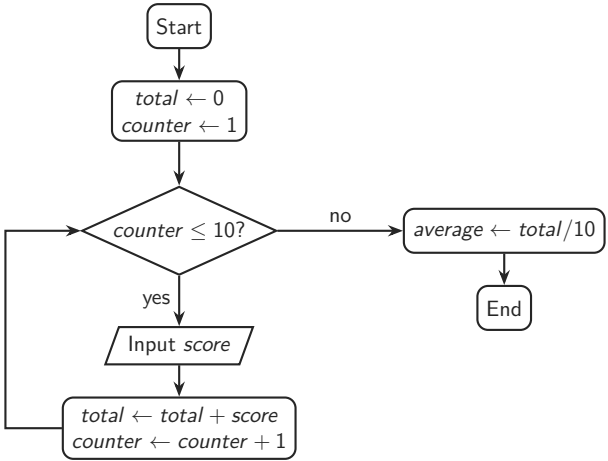
```
Set total to 0
Set student_counter to 0
While student_counter < 10
    Input score
    Set total to total + score
    Set student_counter to student_counter + 1
EndWhile
Set class_average to total / student_counter
```

Can you tell my first language was C?

I don't like magic numbers in the code. You might change how many students you count, or let a user terminate early, and this would break the assumption that you have 10 students.

Another reason to do this is that the `range` command in Python will give you numbers less than the argument.

Discussion 1: The Control Flow



Discussion 2: Fill in the Decisions

```
Set passes to 0; set failures to 0
Set student_counter to 1
While student_counter <= 10
    Input score
    --------------------------
        ----------------------
    Else
        ----------------------
    EndIf
    Set student_counter to student_counter + 1
EndWhile
-----------------------------
```

Assume the pass mark is available as `pass_mark`.

Discussion 2: One Solution

```
If score >= pass_mark
    Set passes to passes + 1
Else
    Set failures to failures + 1
EndIf
...
Print passes
Print failures
```

Check

After every iteration, $passes + failures = student\_counter - 1$.

Discussion 3: FizzBuzz

For each integer from 1 through 20:

- print `FizzBuzz` if it is divisible by both 3 and 5;
- otherwise print `Fizz` if divisible by 3;
- otherwise print `Buzz` if divisible by 5;
- otherwise print the number.

Discussion 3: FizzBuzz

For each integer from 1 through 20:

- print `FizzBuzz` if it is divisible by both 3 and 5;
- otherwise print `Fizz` if divisible by 3;
- otherwise print `Buzz` if divisible by 5;
- otherwise print the number.

Hint: Order matters!

Discussion 3: FizzBuzz

For each integer from 1 through 20:

- otherwise print `Fizz` if divisible by 3;
- otherwise print `Buzz` if divisible by 5;
- otherwise print the number.

When given as an interview problem, this step is often left out, you're supposed to ask.

Discussion 3: Pseudocode

```
For num from 1 to 20
    If num MOD 15 = 0
        Print "FizzBuzz"
    Else if num MOD 3 = 0
        Print "Fizz"
    Else if num MOD 5 = 0
        Print "Buzz"
    Else
        Print num
    EndIf
EndFor
```

Discussion 3: Full Python Solution

```
for num in range(1, 21):
    if num % 15 == 0:
        print("FizzBuzz")
    elif num % 3 == 0:
        print("Fizz")
    elif num % 5 == 0:
        print("Buzz")
    else:
        print(num)
```

Discussion 3: Modulo in Math Notation

If $a \bmod n = r$, then dividing a by n leaves remainder r.

$$15 \bmod 3 = 0 \qquad 15 \bmod 5 = 0$$
$$14 \bmod 3 = 2 \qquad 14 \bmod 5 = 4$$

Number-theory notation

$$n \equiv 0 \pmod 3 \quad \text{means} \quad 3 \mid n$$
$$n \equiv 0 \pmod 5 \quad \text{means} \quad 5 \mid n$$
$$n \equiv 0 \pmod{15} \quad \text{means} \quad 15 \mid n$$

So FizzBuzz starts by testing whether $n \equiv 0 \pmod{15}$ before the separate tests modulo 3 and 5.

Can we do it with two if statements?

```
for num in range(1, 20):
     if num % 3 == 0:
         print("Fizz", end="")
     if num % 5 == 0:
         print("Buzz", end="")
     if (not num % 3 == 0) and (not num % 5 ==0):
         print(num, end="")
     print("!")
```

Tricky because Python automatically puts newline after `print` command.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`c = 10`: valid assignment; now `c` stores 10.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`7 = a`: syntax error because the left side is not assignable.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a = d`: name error because `d` has not been defined.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a = c + 1`: valid assignment; a becomes 11 if line 1 already ran.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a + c = c`: syntax error because an expression cannot be assigned to.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`3 + a`: valid expression (14), but a script prints nothing unless asked.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`7up = 10`: syntax error because identifiers cannot start with a digit.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`import = 1003`: syntax error because `import` is a keyword.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`b = math.pi * c`: name error unless `math` was imported first, otherwise 31.415...

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`int = 500`: legal, but it shadows the built-in `int`.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a ** 3`: valid expression if `a` exists (1331); no printed output in a script.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a, b, c = c, 1, a`: valid simultaneous assignment if the names already exist:
$a = 10; b = 1; c = 11$

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`b, c, a = a, b`: value error because three variables expect only two values.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`c = b = a = 7`: valid chained assignment: $c = 7, b = 7, a = 7$

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`print(A)`: name error because Python is case-sensitive.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`print("b*b + a*a = c*c")`: prints the literal text, not a calculation.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`print('A')`: prints the character `A`.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`print("c" = 1)`: syntax error because assignment is not a function argument.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

c = 10: valid assignment; now c stores 10.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`7 = a`: syntax error because the left side is not assignable.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a = d`: name error because `d` has not been defined.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a = c + 1`: valid assignment; a becomes 11 if line 1 already ran.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a + c = c`: syntax error because an expression cannot be assigned to.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`3 + a`: valid expression (14), but a script prints nothing unless asked.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`7up = 10`: syntax error because identifiers cannot start with a digit.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`import = 1003`: syntax error because `import` is a keyword.

Discussion 4: Predict Before Running (1–9)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`b = math.pi * c`: name error unless `math` was imported first, otherwise 31.415...

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`int = 500`: legal, but it shadows the built-in `int`.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a ** 3`: valid expression if `a` exists (1331); no printed output in a script.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`a, b, c = c, 1, a`: valid simultaneous assignment if the names already exist:
$a = 10; b = 1; c = 11$

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`b, c, a = a, b`: value error because three variables expect only two values.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`c = b = a = 7`: valid chained assignment: $c = 7, b = 7, a = 7$

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`print(A)`: name error because Python is case-sensitive.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`print("b*b + a*a = c*c")`: prints the literal text, not a calculation.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`print('A')`: prints the character `A`.

Discussion 4: Predict Before Running (10–18)

Assume the lines are executed in this order.

```
c = 10
7 = a
a = d
a = c + 1
a + c = c
3 + a
7up = 10
import = 1003
b = math.pi * c
```

```
int = 500
a ** 3
a, b, c = c, 1, a
b, c, a = a, b
c = b = a = 7
print(A)
print("b*b + a*a = c*c")
print('A')
print("c" = 1)
```

`print("c" = 1)`: syntax error because assignment is not a function argument.

Discussion 5: Class Percentages

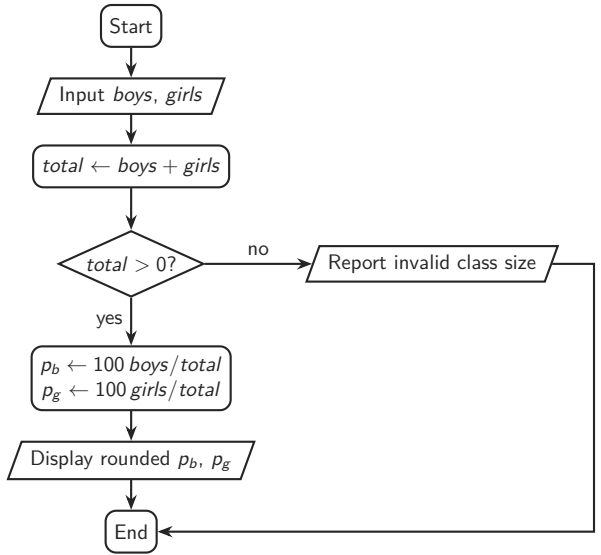
Ask for the numbers of boys and girls, then display each percentage.

Sample interaction

```
Enter the number of boys:  65
Enter the number of girls:  77
Boys:  46%    Girls:  54%
```

Design the algorithm and flowchart first.

Discussion 5: Flowchart



Discussion 5: Python

```
boys = int(input("Enter the number of boys: "))
girls = int(input("Enter the number of girls: "))
total = boys + girls

if total <= 0:
    print("The class must contain at least one student.")
else:
    percent_boys = boys / total
    percent_girls = girls / total
    print("Boys:", format(percent_boys, ".0%"))
    print("Girls:", format(percent_girls, ".0%"))
```

Discussion 6: Route Planning

Start at the hotel, visit every attraction once, and return to the hotel.

- We want a short route, but do not need the absolute optimum.
- Assume a distance matrix already exists.
- Propose a logical, easy-to-follow heuristic in pseudocode.

Discussion 6: Route Planning

Start at the hotel, visit every attraction once, and return to the hotel.

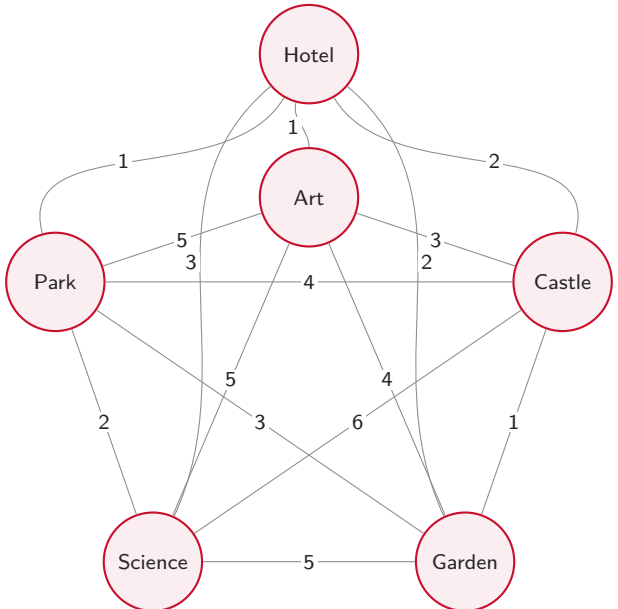
- We want a short route, but do not need the absolute optimum.
- Assume a distance matrix already exists.
- Propose a logical, easy-to-follow heuristic in pseudocode.

Propose a solution to the above problem and describe it using pseudocode. You do NOT need to find the absolute best route, ignore data structure details; assume that the distance matrix has already been created.

Discussion 6: Distances Between Attractions

	Hotel	Art	Castle	Garden	Science	Park
Hotel	0	1	2	2	3	1
Museum of Art	1	0	3	4	5	5
Historical Castle	2	3	0	1	6	4
Botanical Garden	2	4	1	0	5	3
Science Center	3	5	6	5	0	2
City Park	1	5	4	3	2	0

Discussion 6: The Distance Graph



Discussion 6: Nearest-Neighbour Heuristic

```
Set current_location to Hotel
Mark every attraction unvisited; set route to [Hotel]
While an unvisited attraction remains
    Set nearest_distance to infinity
    For each attraction in unvisited
        Set distance to distance_matrix[current_location][attraction]
        If distance < nearest_distance
            Set nearest_distance to distance
            Set nearest_attraction to attraction
        EndIf
    EndFor
    Visit and mark nearest_attraction; append it to route
    Set current_location to nearest_attraction
EndWhile
Append Hotel to route; display route
```

Repeat until we've visited everything

Discussion 6: Nearest-Neighbour Heuristic

```
Set current_location to Hotel
Mark every attraction unvisited; set route to [Hotel]
While an unvisited attraction remains
    Set nearest_distance to infinity
    For each attraction in unvisited
        Set distance to distance_matrix[current_location][attraction]
        If distance < nearest_distance
            Set nearest_distance to distance
            Set nearest_attraction to attraction
        EndIf
    EndFor
    Visit and mark nearest_attraction; append it to route
    Set current_location to nearest_attraction
EndWhile
Append Hotel to route; display route
```

Check every currently unvisited attraction.

Discussion 6: Nearest-Neighbour Heuristic

```
Set current_location to Hotel
Mark every attraction unvisited; set route to [Hotel]
While an unvisited attraction remains
    Set nearest_distance to infinity
    For each attraction in unvisited
        Set distance to distance_matrix[current_location][attraction]
        If distance < nearest_distance
            Set nearest_distance to distance
            Set nearest_attraction to attraction
        EndIf
    EndFor
    Visit and mark nearest_attraction; append it to route
    Set current_location to nearest_attraction
EndWhile
Append Hotel to route; display route
```

Update the best candidate only when we find a shorter edge.

Discussion 6: Nearest-Neighbour Heuristic

```
Set current_location to Hotel
Mark every attraction unvisited; set route to [Hotel]
While an unvisited attraction remains
    Set nearest_distance to infinity
    For each attraction in unvisited
        Set distance to distance_matrix[current_location][attraction]
        If distance < nearest_distance
            Set nearest_distance to distance
            Set nearest_attraction to attraction
        EndIf
    EndFor
    Visit and mark nearest_attraction; append it to route
    Set current_location to nearest_attraction
EndWhile
Append Hotel to route; display route
```

Save that stop.

Discussion 6: Nearest-Neighbour Heuristic

```
Set current_location to Hotel
Mark every attraction unvisited; set route to [Hotel]
While an unvisited attraction remains
    Set nearest_distance to infinity
    For each attraction in unvisited
        Set distance to distance_matrix[current_location][attraction]
        If distance < nearest_distance
            Set nearest_distance to distance
            Set nearest_attraction to attraction
        EndIf
    EndFor
    Visit and mark nearest_attraction; append it to route
    Set current_location to nearest_attraction
EndWhile
Append Hotel to route; display route
```

Save choice and repeat.

Discussion 6: Ties and Trade-offs

Hotel–Art and Hotel–Park are both 1 km.

- Use a deterministic tie-breaker: first in the list, alphabetical order, or lowest ID.
- The heuristic is fast and easy to explain.
- It can miss the globally shortest tour because it never revisits an earlier choice.

One deterministic run

Hotel $\rightarrow$ Art $\rightarrow$ Castle $\rightarrow$ Garden $\rightarrow$ Park $\rightarrow$ Science $\rightarrow$ Hotel
Total distance: $1 + 3 + 1 + 3 + 2 + 3 = 13$ km.

Discussion 6: Ties and Trade-offs

Hotel–Art and Hotel–Park are both 1 km.

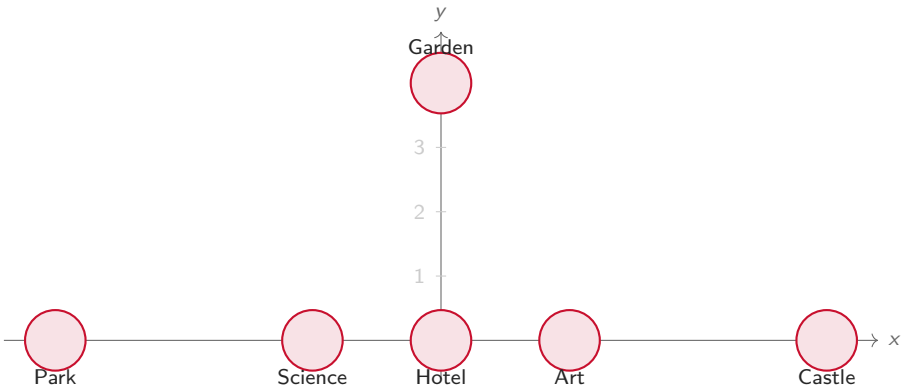
- Use a deterministic tie-breaker: first in the list, alphabetical order, or lowest ID.
- The heuristic is fast and easy to explain.
- It can miss the globally shortest tour because it never revisits an earlier choice.

One deterministic run

Hotel $\rightarrow$ Art $\rightarrow$ Castle $\rightarrow$ Garden $\rightarrow$ Park $\rightarrow$ Science $\rightarrow$ Hotel
Total distance: $1 + 3 + 1 + 3 + 2 + 3 = 13$ km.

Can you think of where the Greedy approach might fail?

Discussion 6: A Counterexample in the Plane



Points: Hotel $(0, 0)$, Art $(1, 0)$, Castle $(3, 0)$, Garden $(0, 4)$, Science $(-1, 0)$, Park $(-3, 0)$.

Discussion 6: Counterexample Distance Matrix

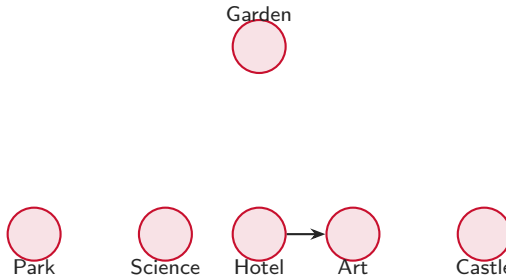
	Hotel	Art	Castle	Garden	Science	Park
Hotel	0.00	1.00	3.00	4.00	1.00	3.00
Art	1.00	0.00	2.00	4.12	2.00	4.00
Castle	3.00	2.00	0.00	5.00	4.00	6.00
Garden	4.00	4.12	5.00	0.00	4.12	5.00
Science	1.00	2.00	4.00	4.12	0.00	2.00
Park	3.00	4.00	6.00	5.00	2.00	0.00

These are Euclidean distances rounded to two decimal places.

Discussion 6: Walking Through the Greedy Choice

Assume ties are broken alphabetically, so from Hotel the algorithm chooses Art instead of Science.

- Hotel $\rightarrow$ Art because both Art and Science are distance 1.00.

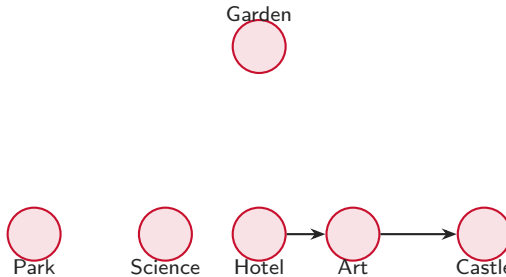


Running total grows as the path commits to each local choice.

Discussion 6: Walking Through the Greedy Choice

Assume ties are broken alphabetically, so from Hotel the algorithm chooses Art instead of Science.

- Hotel $\rightarrow$ Art because both Art and Science are distance 1.00.
- Art $\rightarrow$ Castle because 2.00 is the smallest remaining distance.

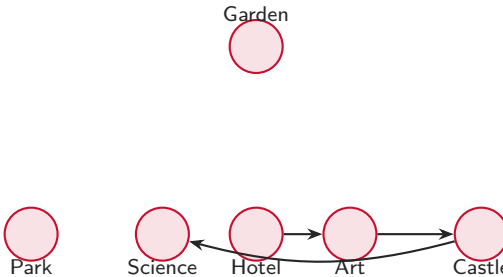


Running total grows as the path commits to each local choice.

Discussion 6: Walking Through the Greedy Choice

Assume ties are broken alphabetically, so from Hotel the algorithm chooses Art instead of Science.

- Hotel $\rightarrow$ Art because both Art and Science are distance 1.00.
- Art $\rightarrow$ Castle because 2.00 is the smallest remaining distance.
- Castle $\rightarrow$ Science because 4.00 beats 5.00 and 6.00.

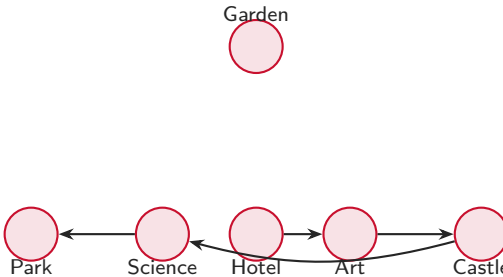


Running total grows as the path commits to each local choice.

Discussion 6: Walking Through the Greedy Choice

Assume ties are broken alphabetically, so from Hotel the algorithm chooses Art instead of Science.

- Hotel → Art because both Art and Science are distance 1.00.
- Art → Castle because 2.00 is the smallest remaining distance.
- Castle → Science because 4.00 beats 5.00 and 6.00.
- Science → Park because 2.00 is smallest.



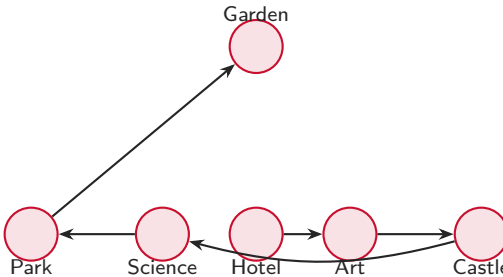
Running total grows as the path commits to each local choice.

Discussion 6: Walking Through the Greedy Choice

Assume ties are broken alphabetically, so from Hotel the algorithm chooses Art instead of Science.

- Hotel → Art because both Art and Science are distance 1.00.
- Art → Castle because 2.00 is the smallest remaining distance.
- Castle → Science because 4.00 beats 5.00 and 6.00.
- Science → Park because 2.00 is smallest.
- Park → Garden is forced, with distance 5.00.

Running total grows as the path commits to each local choice.

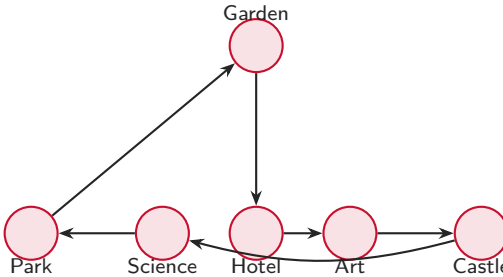


Discussion 6: Walking Through the Greedy Choice

Assume ties are broken alphabetically, so from Hotel the algorithm chooses Art instead of Science.

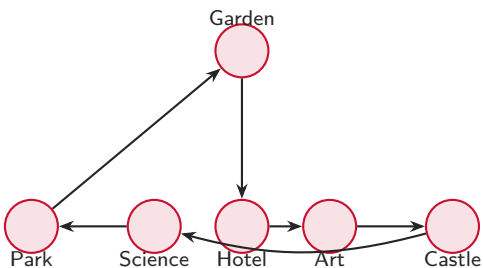
- Hotel → Art because both Art and Science are distance 1.00.
- Art → Castle because 2.00 is the smallest remaining distance.
- Castle → Science because 4.00 beats 5.00 and 6.00.
- Science → Park because 2.00 is smallest.
- Park → Garden is forced, with distance 5.00.
- Garden → Hotel closes the tour.

Total greedy length:
$1.00 + 2.00 + 4.00 + 2.00 + 5.00 + 4.00 = 18.00$.



Discussion 6: Greedy vs. Better Tour

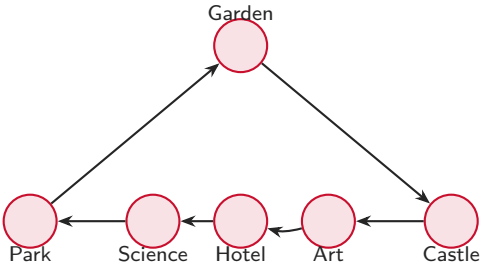
Greedy choice



Hotel → Art → Castle → Science → Park → Garden → Hotel

18.00

Shorter tour



Hotel → Science → Park → Garden → Castle → Art → Hotel

16.00

Discussion 6: Python Code (1/2)

```
distances = {
    "Hotel": {"Art": 1, "Castle": 2, "Garden": 2, "Science": 3, "Park": 1},
    "Art": {"Hotel": 1, "Castle": 3, "Garden": 4, "Science": 5, "Park": 5},
    "Castle": {"Hotel": 2, "Art": 3, "Garden": 1, "Science": 6, "Park": 4},
    "Garden": {"Hotel": 2, "Art": 4, "Castle": 1, "Science": 5, "Park": 3},
    "Science": {"Hotel": 3, "Art": 5, "Castle": 6, "Garden": 5, "Park": 2},
    "Park": {"Hotel": 1, "Art": 5, "Castle": 4, "Garden": 3, "Science": 2},
}
```

Discussion 6: Python Code (2/2)

```
route = ["Hotel"]
unvisited = {"Art", "Castle", "Garden", "Science", "Park"}
current = "Hotel"

while unvisited:
    nearest_distance = float("inf")
    nearest_attraction = None

    for place in unvisited:
        distance = distances[current][place]
        if distance < nearest_distance:
            nearest_distance = distance
            nearest_attraction = place

    route.append(nearest_attraction)
    unvisited.remove(nearest_attraction)
    current = nearest_attraction

route.append("Hotel")
print(" -> ".join(route))
```

Takeaways

- Initialize variables
- Don't overcomplicate expressions
- Good enough isn't always optimal
- Flowcharts expose decisions and loops before code obscures them.